

ĐẠI HỌC QUỐC GIA, THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



KIẾN TRÚC MÁY TÍNH (CO2008)

BÀI TẬP LỚN

KIẾN TRÚC TẬP LỆNH

Giảng viên hướng dẫn: Nguyễn Xuân Minh

Sinh viên thực hiện: Nguyễn Đức Thịnh - 2313284 - L01
Lê Triều Kha - 2311388 - L05

TP Hồ Chí Minh, Tháng 11/2024



Mục lục

1	Giới thiệu đề tài	2
1.1	Đề tài	2
1.2	Giải thuật Selection sort	2
2	Selection sort và cách hiện thức	2
2.1	Mô tả giải thuật	2
2.2	Mã giả giải thuật	3
3	Thống kê số lượng lệnh sử dụng trong chương trình	3
4	Tính thời gian chạy của chương trình	4
5	Kết quả kiểm thử	4

1 Giới thiệu đề tài

1.1 Đề tài

Đề 5. Selection sort số nguyên.

Viết chương trình sắp xếp dãy số nguyên có 10 phần tử dùng giải thuật Selection sort. Yêu cầu xuất ra màn hình các bước có thay đổi thứ tự trong dãy. Dữ liệu đầu vào đọc từ file lưu trữ dạng nhị phân trên đĩa INT10.BIN (10 phần tử x 4 bytes = 40 bytes).

1.2 Giải thuật Selection sort

Giải thuật Selection Sort là một thuật toán sắp xếp đơn giản và dễ hiểu, nhưng không phải lúc nào cũng hiệu quả đối với các mảng lớn. Ý tưởng chính của Selection Sort là tìm phần tử nhỏ nhất trong phần chưa được sắp xếp của mảng và đổi chỗ nó với phần tử đầu tiên của phần chưa sắp xếp đó. Quá trình này được lặp lại cho đến khi mảng được sắp xếp hoàn toàn.

Cụ thể, quá trình Selection Sort được thực hiện như sau:

1. **Khởi tạo:** Chia mảng thành hai phần: một phần đã sắp xếp và một phần chưa sắp xếp. Ban đầu, phần đã sắp xếp là rỗng, và phần chưa sắp xếp là toàn bộ mảng.
2. **Tìm phần tử nhỏ nhất:** Chọn phần tử nhỏ nhất trong phần chưa sắp xếp của mảng. Phần tử này sẽ là phần tử cần đổi chỗ với phần tử đầu tiên của phần chưa sắp xếp.
3. **Đổi chỗ (Swap):** Sau khi tìm được phần tử nhỏ nhất, thực hiện đổi chỗ phần tử này với phần tử đầu tiên của phần chưa sắp xếp. Phần tử nhỏ nhất được đưa vào vị trí chính xác của nó trong mảng đã sắp xếp.
4. **Lặp lại:** Tiến hành lặp lại bước 2 và 3 với phần còn lại của mảng (tức là bỏ qua phần tử đầu tiên đã được sắp xếp). Quá trình này tiếp tục cho đến khi không còn phần tử nào trong phần chưa sắp xếp.
5. **Kết thúc:** Mảng được sắp xếp hoàn toàn khi tất cả các phần tử đều được đặt vào đúng vị trí của chúng trong mảng.

Điểm mạnh của Selection sort là nó đơn giản, dễ hiểu, không cần phụ thuộc bộ nhớ phụ. Độ phức tạp thời gian của Selection sort là $O(n^2)$ trong cả trường hợp tốt và xấu trong đó n là số phần tử của mảng. Tuy nhiên, điểm yếu của Selection sort là hiệu suất kém khi xử lý mảng lớn, vì độ phức tạp $O(n^2)$.

2 Selection sort và cách hiện thức

2.1 Mô tả giải thuật

- **Input:** Mảng cần được sắp xếp, arr.
- **Output:** Mảng arr đã được sắp xếp tăng dần.
- **Mô tả:**
 - Nếu độ dài của mảng arr là 0 hoặc 1, tức là mảng đã được sắp xếp hoàn tất (vì không có gì để sắp xếp hoặc chỉ có một phần tử), trả về mảng arr mà không cần thay đổi.

- Nếu độ dài của mảng `arr` lớn hơn 1, thực hiện các bước sau:
 - * Lặp qua mảng từ vị trí đầu tiên đến vị trí cuối cùng của mảng, tìm phần tử nhỏ nhất trong phần chưa được sắp xếp.
 - * Đổi chỗ phần tử nhỏ nhất tìm được với phần tử đầu tiên của phần chưa được sắp xếp.
 - * Sau mỗi lần lặp, phần tử nhỏ nhất đã được đưa vào đúng vị trí của nó trong mảng đã sắp xếp.
- Tiến hành các bước trên cho tất cả các phần tử trong mảng, mỗi lần giảm kích thước phần chưa sắp xếp đi 1 phần tử.
- Khi không còn phần tử nào trong phần chưa sắp xếp, mảng sẽ hoàn toàn được sắp xếp.

2.2 Mã giả giải thuật

Dưới đây là mã giả của Selection sort.

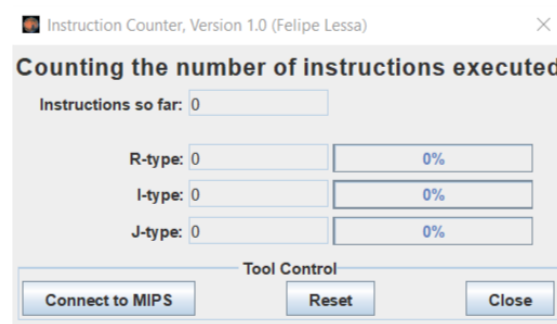
```
Input: Array data of size count  
Output: Sorted array data  
1 Function SelectionSort(data, count):  
2   current  $\leftarrow$  0;  
3   while current < count - 1 do  
4     smallest  $\leftarrow$  current;  
5     walker  $\leftarrow$  current + 1;  
6     while walker < count do  
7       if data[walker] < data[smallest] then  
8         smallest  $\leftarrow$  walker;  
9       end  
10      walker  $\leftarrow$  walker + 1;  
11    end  
12    swap(current, smallest);  
13    current  $\leftarrow$  current + 1;  
14  end
```

3 Thống kê số lượng lệnh sử dụng trong chương trình

Để thống kê số lượng lệnh sử dụng trong chương trình, chúng ta sử dụng công cụ Instruction Counter để đếm số lệnh trong chương trình:

Trong đó:

- Instructions so far: là tổng số lệnh thực thi
- R-type: là số lệnh loại R
- I-type: là số lệnh loại I
- J-type: là số lệnh loại J



Hình 1: Công cụ Instruction Counter.

4 Tính thời gian chạy của chương trình

Sử dụng công thức sau để tính toán thời gian chạy của chương trình

$$\text{CPU time} = \frac{\text{CPU Clock cycles}}{\text{Clock rate}} = \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock rate}}$$

Trong đó:

- CPU time: là thời gian xử lý của chương trình (không tính thời gian giao tiếp I/O, thời gian chờ...).
- CPU Clock cycles: là tổng số chu kỳ thực thi.
- Instruction Count: là tổng số lệnh thực thi của chương trình.
- CPI (cycle per instruction): là số chu kỳ thực thi trên một lệnh.
- Clock rate: là số chu kỳ thực thi trên một giây hay còn gọi là tần số.

Tùy vào dữ liệu đầu vào và nội dung mảng mà chương trình sẽ có tổng số lệnh, loại lệnh và thời gian chạy khác nhau.

5 Kết quả kiểm thử

Dưới đây là kết quả đánh giá trên 10 test case, tần số được cho là 1 GHz, cho CPI = 1.

Thứ tự	Mảng ban đầu	Mảng đã sắp xếp	R	I	J	IC	Time (μs)
1	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	376	426	66	868	0.868
2	9, 8, 7, 6, 5, 4, 3, 2, 1, 0	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	716	916	126	1758	1.758
3	55, 24, 58, 34, 19, 72, 80, 4, 57, 97	4, 19, 24, 34, 55, 57, 58, 72, 80, 97	828	1088	150	2066	2.066



Thứ tự	Mảng ban đầu	Mảng đã sắp xếp	R	I	J	IC	Time (μs)
4	35, 32, 84, 71, 94, 99, 2, 86, 92, 10	2, 10, 32, 35, 71, 84, 86, 92, 94, 99	893	1183	162	2238	2.238
5	28, 77, 16, 44, 72, 92, 23, 4, 82, 99	4, 16, 23, 28, 44, 72, 77, 82, 92, 99	827	1087	150	2064	2.064
6	801, 316, 479, 35, 216, 185, 705, 243, 873, 553	35, 185, 216, 243, 316, 479, 553, 705, 801, 873	892	1182	162	2236	2.236
7	195, 36, 693, 371, 110, 256, 210, 683, 845, 375	36, 110, 195, 210, 256, 371, 375, 683, 693, 845	891	1181	162	2234	2.234
8	-28, -77, -16, 44, 72, 92, -23, -4, -82, -99	-99, -82, -77, -28, -23, -16, -4, 44, 72, 92	958	1278	174	2410	2.410
9	-45, -94, 19, 52, 77, -78, 45, 69, -93, -67	-94, -93, -78, -67, -45, 19, 45, 52, 69, 77	828	1088	150	2066	2.066
10	-496, -249, 55, 19, -213, 324, 94, -481, 178, 413	-496, -481, -249, -213, 19, 55, 94, 178, 324, 413	699	899	126	1724	1.724

Bảng 1: Bảng thống kê kết quả sắp xếp.



Nguồn tham khảo

- [1] MIPS Technologies, Inc *MIPS32™ Architecture For Programmers Volume II: The MIPS32™ Instruction Set*. 2003.
- [2] Phạm Quốc Cường, *Kiến trúc máy tính*, Nxb. Đại học Quốc gia TP HCM, 2021
- [3] Hợp ngữ MIPS: <https://vietcodes.github.io/algo/mips>.
- [4] Selection sort: https://en.wikipedia.org/wiki/Selection_sort.